

Prompt Engineering, In-context Learning & LangChain

Complete Study Notes · Generative AI Reference

Part	Topic	Colour Theme
1	In-context Learning & Prompt Engineering Fundamentals	Indigo
2	LangChain — Framework, Benefits & Applications	Emerald
3	Advanced Prompt Engineering Methods	Saffron

PART 1 — In-context Learning & Prompt Engineering Fundamentals

1.1 What is In-context Learning?

In-context learning (ICL) is a specific method of prompt engineering where demonstrations of the task are provided to the model as part of the prompt in natural language. Crucially, it does **not require additional training or fine-tuning** of the model. A new task is learned from a small set of examples presented within the context (prompt) at inference time — the moment the model is actually used.

Advantages

- No fine-tuning required — no need to update model weights.
- Drastically reduces time and compute resources needed to adapt LLMs for specific tasks.
- Improves performance on new tasks quickly.
- Flexible — works across many task types simply by changing the prompt.

Disadvantages

- Constrained by context window size — only so much can fit in a prompt.
- Not suited for complex tasks requiring deep reasoning over many steps.
- May not match fine-tuning performance on highly specialised tasks.
- Complex tasks may still need gradient-based training (weight adjustment).

Key distinction: In-context learning happens at inference time with no weight updates. Fine-tuning changes the model's parameters using labelled data during a training phase. ICL is faster and cheaper; fine-tuning is more powerful for specialised domains.

1.2 What is a Prompt?

A **prompt** is an instruction or input given to an LLM, designed to guide it toward performing a specific task or generating a desired output. Think of it as the question or command you give the AI.

The Two Core Components of a Prompt

Instructions

Context

Clear, direct commands that tell the AI what to do. They must be specific so the LLM understands the exact task required.

Example: 'Classify the following customer review into neutral, negative, or positive sentiment.'

The background information or scenario that helps the LLM make sense of the instruction. Can include data, parameters, or relevant details that shape the AI's response.

Example: 'This review is for a recently launched product.'

1.3 The Four Elements of a Well-Structured Prompt

A complete, well-structured prompt typically contains four key elements. The example below illustrates all four working together for a sentiment analysis task:

No.	Element	Purpose	Example
1	Instructions	Tell the LLM exactly what to do.	<i>Classify the following customer review into neutral, negative, or positive sentiment.</i>
2	Context	Provide the background scenario to help the model interpret the task correctly.	<i>This review is part of feedback for a recently launched product.</i>
3	Input Data	The actual content the LLM will process to perform the task.	<i>'The product arrived late but the quality exceeded my expectations.'</i>
4	Output Indicator	A clear marker that signals where the LLM should deliver its response.	<i>Sentiment: [LLM appends its classification here]</i>

1.4 What is Prompt Engineering?

Prompt engineering is the specialised process of designing and refining the questions, commands, or statements used to interact with AI systems — particularly LLMs. The goal is not just to ask a question, but to ask it in the **best way possible** to elicit the most accurate, relevant, and useful response.

Why Prompt Engineering Matters

- **Boosts effectiveness and accuracy** — directly influences how well and how accurately LLMs perform.
- **Ensures relevance** — enables LLMs to generate precise responses perfectly suited to context.
- **Meets user expectations** — clearer prompts lead to fewer misunderstandings and better outputs.
- **Eliminates continual fine-tuning** — the model adapts within its context rather than requiring retraining.
- **Enables cross-domain use** — the same model can be guided for customer service, research, coding, and more.

Example — Open-ended creative prompt: Prompt: 'The wind is' Response: 'Blowing gently through the trees, whispering secrets and stories to anyone who cares to listen.' This illustrates how even a simple prompt can guide an LLM to produce imaginative, detailed, and contextually rich output.

Part 1 — Key Takeaways

- 1 In-context learning provides task demonstrations inside the prompt — no fine-tuning or weight updates needed.
- 2 It is fast and resource-efficient but limited by context window size and task complexity.
- 3 A prompt is an instruction or input that guides an LLM toward a specific task or output.
- 4 Every prompt has two core components: Instructions (what to do) and Context (background information).
- 5 A well-structured prompt has four elements: Instructions, Context, Input Data, and Output Indicator.
- 6 Prompt engineering is the craft of designing prompts for maximum accuracy, relevance, and usefulness.

PART 2 — LangChain: Framework, Benefits & Applications

2.1 What is LangChain?

LangChain is an **open-source Python framework** that streamlines the development of applications powered by large language models (LLMs). It provides developers with ready-made components and interfaces to integrate LLMs into their AI applications more efficiently.

The name 'LangChain' reflects its core mechanism: it **chains together** retrieval, extraction, processing, and generation operations across large amounts of text from multiple sources.

Core Capabilities

- Pinpoint relevant information in large text sources such as research papers, legal documents, or knowledge bases.
- Respond to complex prompts by retrieving data and generating coherent, contextually accurate summaries.
- Chain multiple operations together — retrieve → extract → process → generate — in a single pipeline.
- Work with vector databases for efficient semantic search and information retrieval.
- Support agents that can perform multi-step, complex tasks across different domains.

2.2 Key Benefits of LangChain

Modularity	LangChain's design allows developers to piece together different components like building blocks. Components are reusable across projects, reducing development time and effort. Teams can swap, update, or test individual components independently without rewriting the entire pipeline.
Extensibility	The framework's extensible architecture makes it easy to add new features, adapt existing components, and integrate with external systems. Changes to one part of the codebase are minimal and localised. This future-proofs applications as LLM capabilities evolve.
Decomposition	LangChain mimics human problem-solving by breaking complex queries or tasks into smaller, manageable steps. This step-by-step approach enables the model to make accurate inferences from context, resulting in precise and relevant responses even for multi-part questions.
Vector DB Integration	LangChain integrates natively with vector databases (e.g. Pinecone, Chroma, FAISS) for efficient semantic search and similarity-based retrieval. Applications gain quick access to the most relevant information within massive datasets, dramatically improving response quality for knowledge-intensive tasks.

2.3 Practical Applications of LangChain

Content Summarisation

Summarise articles, reports, and complex documents automatically. Particularly useful for deciphering dense legal contracts, academic papers, or policy documents. Users stay informed without reading every word.

Data Extraction

Extract key statistics, entities, and insights from large reports or datasets. Converts unstructured text into actionable, structured information — turning raw data into decisions.

Question & Answer Systems

Build intelligent Q&A; systems for customer support and knowledge bases. These systems provide contextually relevant answers based on the entire conversation history, not just a single query — enabling follow-up clarification chains.

Automated Content Generation

Automate routine writing tasks: drafting emails, creating technical documentation, brainstorming ideas, or generating product descriptions. Reduces manual effort and maintains consistency in tone and style.

2.4 Working with Other Data Types

While primarily designed for text, LangChain can work with **images, audio, and video** by leveraging external libraries and models. For example, speech-to-text models can convert audio to text before LangChain processes it. Its integration with vector databases enables the use of **embeddings** — numerical representations of semantic meaning — generated from any data type, enabling powerful similarity searches across modalities.

What are Embeddings? An embedding is a numerical vector that captures the semantic meaning of a piece of data. Similar content produces similar vectors, enabling 'semantic search' — finding related information even when exact words don't match. LangChain uses these embeddings with vector databases to retrieve the most contextually relevant information for any query.

Part 2 — Key Takeaways

1

LangChain is an open-source Python framework that chains retrieval, extraction, processing, and generation operations for LLM apps.

2

Its four key benefits are: modularity, extensibility, decomposition, and vector database integration.

3

Practical uses include: document summarisation, data extraction, Q&A; systems, and automated content generation.

4

LangChain can work beyond text — handling images, audio, and video via external libraries and embedding models.

5

Vector database integration enables semantic search — finding relevant information by meaning, not just keyword matching.

PART 3 — Advanced Prompt Engineering Methods

3.1 Overview of Advanced Prompting Techniques

Beyond basic prompting, several advanced techniques help unlock more powerful and reliable behaviour from LLMs. Each technique is suited to different scenarios depending on available examples, task complexity, and required accuracy.

Technique	Examples Provided	Best Used When
Zero-shot	None	Task is simple or the model has strong prior knowledge
One-shot	1 example	Format needs demonstrating with minimal overhead
Few-shot	2–5 examples	Pattern generalisation from limited labelled data
Chain-of-Thought	Reasoning steps	Multi-step reasoning, maths, logic, complex problems
Self-consistency	Multiple outputs	High-stakes answers requiring reliability verification

3.2 Zero-shot Prompting

A zero-shot prompt instructs the LLM to perform a task **without any prior specific examples**. The model relies entirely on its pre-trained knowledge and the clarity of the instruction.

Example: Prompt: 'Classify the following statement as True or False: The Eiffel Tower is located in Berlin.'
Response: 'False — The Eiffel Tower is located in Paris, France.' The model applies world knowledge without any examples to guide it.

When to use: Simple fact-based tasks, classification with clear categories, tasks the model is already well-trained on. Not ideal for highly specialised or ambiguous tasks.

3.3 One-shot Prompting

A one-shot prompt gives the LLM **a single example** to demonstrate the desired format or behaviour. The model learns the pattern from that one instance and applies it to the new task.

Example: Prompt: 'Translate English to French. English: How is the weather today? → French: Comment est la météo aujourd'hui? English: Where is the nearest supermarket? → French:' Response: 'Où est le supermarché le plus proche?' The single example serves as a format template for the model to follow.

3.4 Few-shot Prompting

Few-shot prompting provides the LLM with **a small set of labelled examples** (typically 2–5) before asking it to perform a similar task. The model generalises from these few instances to handle new, unseen data.

Example — Emotion Classification: 'I just got promoted!' → Joy 'I can't believe I lost my keys again.' → Frustration 'My grandmother passed away today.' → Sadness New input: 'That movie was so scary, I had to cover my eyes.' Response: 'Fear' The three labelled examples teach the model the classification pattern.

When to use: Pattern recognition, classification, format-specific generation where a handful of examples is sufficient to establish the rule.

3.5 Chain-of-Thought (CoT) Prompting

Chain-of-Thought prompting guides the LLM through **complex reasoning step-by-step**, mimicking human thought processes. Instead of jumping to an answer, the model is prompted to show intermediate reasoning steps, which improves accuracy significantly on multi-step problems.

Example — Arithmetic reasoning: Prompt: 'A store had 22 apples. They sold 15, then received 8 more. How many apples are there now? Think step by step.' CoT Response: Step 1: Start with 22 apples. Step 2: Sold 15 → $22 - 15 = 7$ apples remaining. Step 3: Received 8 more → $7 + 8 = 15$ apples. Answer: 15 apples. The step-by-step breakdown both improves accuracy and makes the reasoning transparent.

When to use: Maths problems, logical reasoning, multi-step decision-making, any task where showing working improves reliability.

3.6 Self-consistency

Self-consistency enhances reliability by **generating multiple independent answers** to the same question and selecting the most consistently occurring result. It is particularly effective when combined with Chain-of-Thought prompting.

Example — Age puzzle: Query: 'When I was 6, my sister was half my age. Now I am 70. How old is my sister?' The model produces 3 independent reasoning paths: Path 1: Sister was 3 when I was 6 → difference is 3 years → $70 - 3 = 67$ ✓ Path 2: Half of 6 = 3. Age gap = 3 years. $70 - 3 = 67$ ✓ Path 3: Sister is always 3 years younger. $70 - 3 = 67$ ✓ All three paths agree → Answer: 67 years old (verified by consensus).

When to use: High-stakes outputs, ambiguous problems, situations where a single answer cannot be trusted without verification.

3.7 Tools for Prompt Engineering

Several platforms and tools support the development, testing, and deployment of prompts:

Tool	Provider	Key Features
Playground	OpenAI	Real-time prompt testing against GPT models; parameter adjustment
LangChain	Open-source	Prompt templates, chaining, agent frameworks, vector DB integration
Hugging Face Hub	Hugging Face	Access to thousands of pre-trained models; community prompts
Watsonx.ai	IBM	Enterprise-grade prompt lab; governance and tracking features

3.8 LangChain Prompt Templates

LangChain uses **prompt templates** — predefined, reusable recipes for generating prompts. A template contains: instructions for the model, optional few-shot examples, and placeholders for variable content. This makes prompts consistent, adaptable, and easy to manage.

```
from langchain_core.prompts import PromptTemplate

# Define a reusable template with placeholders
joke_prompt = PromptTemplate.from_template(
    "Tell me a {adjective} joke about {content}"
)

# Fill the placeholders with specific values
prompt = joke_prompt.format(adjective="funny", content="chickens")

# Result: "Tell me a funny joke about chickens"
print(prompt)
```

3.9 Agents in Prompt Applications

An **agent** is an LLM-powered system integrated with tools (like LangChain) that can autonomously perform complex, multi-step tasks across different domains using different prompts. Agents decide which tools to use and in what sequence to achieve a goal.

Q&A; Agents with Sources	Answer questions and cite the specific documents or URLs they retrieved the answer from.
Content Agents	Generate, summarise, or transform written content on demand.
Analytic Agents	Perform data analysis, produce business intelligence reports, and identify trends.
Multilingual Agents	Provide seamless, context-aware translation and communication across languages.

Part 3 — Key Takeaways

- 1 Zero-shot prompting: no examples — the model uses its existing knowledge to perform the task directly.
- 2 One-shot prompting: one example sets the format or pattern for the model to follow.
- 3 Few-shot prompting: 2–5 examples help the model generalise a pattern to new inputs.
- 4 Chain-of-Thought (CoT): breaks complex problems into sequential reasoning steps — greatly improves accuracy.

- 5 Self-consistency: generates multiple independent answers and selects the most consistent result for reliability.
- 6 Prompt engineering tools (LangChain, Playground, Hugging Face) enable testing, templating, and deployment.
- 7 LangChain prompt templates use placeholders for reusable, consistent prompt generation across use cases.
- 8 Agents combine LLMs with tools to autonomously execute multi-step tasks across domains.